

Links con instrucciones levantar contenedor Apache Kafka y crear topic con productor y consumidor

[apache/kafka - Docker Image | Docker Hub](#)

1. En primer lugar debemos Hacer Pull al docker de Apache Kafka para obtener su imagen **mediante la sentencia `docker pull apache/kafka`**

```
C:\Users\david>docker pull apache/kafka
Using default tag: latest
latest: Pulling from apache/kafka
35e38a4b206f: Download complete
b8eeb529f1af: Download complete
643bf8a7c247: Download complete
28bd55152645: Download complete
a723193c2f26: Download complete
729fc64ae8c1: Download complete
f18232174bc9: Download complete
73851e29d6a7: Download complete
b42f712acf6d: Downloading [=====] 72.35MB/135.6MB
c3f73af09931: Download complete
2fa1f65d07a3: Download complete
```

"Hacer pull" en Docker se refiere a la acción de **descargar una imagen de Docker desde un registro (registry)**. El registro más común y público es Docker Hub, pero también existen registros privados.

Los puntos clave sobre docker pull:

- **Propósito:** El objetivo principal de docker pull es obtener una imagen que no está presente localmente en tu máquina. Necesitas la imagen para poder crear y ejecutar contenedores basados en ella.
- **Funcionamiento:** Cuando ejecutas el comando docker pull <nombre_de_la_imagen>, el Docker daemon se conecta al registro configurado (por defecto, Docker Hub) y descarga la imagen especificada, junto con todas las capas y metadatos necesarios.
- **Sintaxis básica:** [docker image pull | Docker Docs](#)

`docker pull [OPTIONS] NOMBRE_DE_LA_IMAGEN[:TAG]@DIGEST]`

- **NOMBRE_DE_LA_IMAGEN:** Es el nombre de la imagen que quieres descargar (ej: ubuntu, nginx, mysql).
- **TAG:** Es una etiqueta que especifica una versión particular de la imagen (ej: 16.04, latest, stable). Si no se especifica un tag, Docker asume :latest.
- **@DIGEST:** Es un identificador único y específico de una versión de la imagen, basado en su contenido. Usar el digest asegura que siempre obtendrás la misma versión, independientemente de los cambios que puedan ocurrir en las etiquetas.
- **OPTIONS:** Permiten modificar el comportamiento del comando pull (ej: autenticación, plataforma).

```
C:\Users\david>docker pull apache/kafka
Using default tag: latest
latest: Pulling from apache/kafka
e9e1a86a14df: Download complete
634d60101085: Download complete
c23d170f77ed: Download complete
a42500819bdc: Download complete
c6a83fedfae6: Download complete
1673c1a2cfa7: Download complete
0b502c76566e: Download complete
a39822f482a5: Download complete
e2d8f56b95eb: Download complete
c4c98316ca78: Download complete
Digest: sha256:c89f315cff967322c5d2021434b32271393cb193aa7ec1d43e97341924e57069
Status: Downloaded newer image for apache/kafka:latest
docker.io/apache/kafka:latest

What's next:
View a summary of image vulnerabilities and recommendations → docker scout quickview apache/kafka
```

2. Ejecutar docker(Run docker) Mediante la sentencia **docker run -d --name broker apache/kafka:latest**

```
C:\Users\david>docker run -d --name broker apache/kafka:latest
db4a076766d89151db6050d9ee6821b70e4cbd27c8fc8e3a99fc5999c76377d3
```

Link a documentación de la instrucción [docker container run | Docker Docs](#)

Docker run es un comando fundamental en Docker que se utiliza para **crear y ejecutar contenedores a partir de una imagen**. En esencia, le dice a Docker que tome una imagen (que puede estar localmente o que Docker intentará descargar de un registro como Docker Hub si no la encuentra) y la ejecute como un proceso aislado llamado contenedor.

los aspectos clave de docker run son:

- **Propósito principal:** Ejecutar una instancia de una imagen, creando así un contenedor en funcionamiento.
- **Qué sucede al ejecutar docker run:**
 1. **Verifica la imagen localmente:** Docker busca la imagen especificada en tu sistema local.
 2. **Descarga la imagen (si es necesario):** Si la imagen no se encuentra localmente, Docker intenta descargarla del registro configurado (por defecto, Docker Hub). Esto es similar a un docker pull automático.
 3. **Crea un nuevo contenedor:** Se crea una instancia ejecutable de la imagen. Esto incluye una capa de escritura sobre la imagen de solo lectura, donde se pueden realizar cambios durante la vida útil del contenedor.
 4. **Configura el contenedor:** Se aplican las opciones que hayas especificado en el comando docker run (por ejemplo, nombre, puertos, volúmenes, variables de entorno).
 5. **Inicia el contenedor:** Se ejecuta el proceso principal definido en la imagen (a través de CMD o ENTRYPOINT en el Dockerfile).
- **Sintaxis básica:**

docker run [OPTIONS] IMAGE [COMMAND] [ARG...]

 - **OPTIONS:** Permiten configurar diversos aspectos del contenedor, como el nombre (--name), mapeo de puertos (-p), montaje de volúmenes (-v), variables de entorno (-e), modo de ejecución (detached -d), etc.
 - **IMAGE:** El nombre de la imagen que se utilizará para crear el contenedor (ej: ubuntu, nginx:latest, mi-imagen:v1.0).
 - **COMMAND y ARG...:** Opcionalmente, puedes especificar un comando diferente al definido en la imagen para que se ejecute dentro del

contenedor, junto con sus argumentos. Esto sobrescribe el CMD definido en el Dockerfile.

3. Abrir la shell in the broker container:

`docker exec --workdir /opt/kafka/bin/ -it broker sh`

```
C:\Users\david>docker exec --workdir /opt/kafka/bin/ -it broker sh
/opt/kafka/bin $ |
```

`docker exec` es un comando de Docker que te permite **ejecutar comandos dentro de un contenedor que ya está en funcionamiento**. Es una herramienta muy útil para interactuar con procesos que se están ejecutando dentro de un contenedor, para depurar problemas, inspeccionar archivos o ejecutar tareas administrativas sin necesidad de detener o reiniciar el contenedor.

Aquí te explico los puntos clave sobre `docker exec`:

- **Propósito principal:** Ejecutar comandos adicionales dentro de un contenedor en ejecución.
- **Cómo funciona:** `docker exec` se conecta al espacio de nombres del contenedor especificado y ejecuta el comando que le indiques dentro de ese entorno aislado. Esto significa que el comando se ejecutará con el contexto del sistema de archivos, la red y otros recursos del contenedor.
- **Sintaxis básica:**
`docker exec [OPTIONS] CONTAINER COMMAND [ARG...]`
 - **OPTIONS:** Permiten configurar cómo se ejecuta el comando dentro del contenedor (ej: modo interactivo, usuario).
 - **CONTAINER:** El nombre o el ID del contenedor en el que quieres ejecutar el comando.
 - **COMMAND:** El comando que quieres ejecutar dentro del contenedor (ej: `ls`, `bash`, `sh`, `cat`, `ps`).
 - **ARG...:** Cualquier argumento adicional que necesite el comando.
- **Opciones comunes:**
 - `-i` o `--interactive`: Mantiene la entrada estándar (`stdin`) abierta, lo que te permite interactuar con el comando (por ejemplo, si ejecutas una shell).
 - `-t` o `--tty`: Asigna una pseudo-terminal (TTY), lo que es útil para comandos interactivos como shells, ya que proporciona funcionalidades como el historial de comandos y la finalización con tabulador.
 - `-it`: La combinación de `-i` y `-t` es muy común para obtener una shell interactiva dentro del contenedor.
 - `-u` o `--user`: Especifica el usuario con el que se ejecutará el comando dentro del contenedor (por defecto, se ejecuta como el usuario definido en la imagen o como `root`).
 - `-w` o `--workdir`: Especifica el directorio de trabajo dentro del contenedor donde se ejecutará el comando.
 - `--privileged`: Otorga privilegios extendidos al comando dentro del contenedor (generalmente no se recomienda por razones de seguridad).

4. Creación de *topics*:

Crear un nuevo *topic* denominado 'pec-topic1-<NombreAlumno>' con dos particiones y con factor de replicación 1.

Link a documentación explicativa [Apache Kafka](#)

Se crea mediante la sentencia `./kafka-topics.sh --bootstrap-server localhost:9092 --create --topic david-carvajal`

```
C:\Users\david>docker exec --workdir /opt/kafka/bin/ -it broker sh
/opt/kafka/bin $ ./kafka-topics.sh --bootstrap-server localhost:9092 --create --topic david-carvajal
Created topic david-carvajal.
/opt/kafka/bin $ |
```

Si quisiéramos comprobar los *topics* que hemos creado, podemos obtener un listado mediante el parámetro `--list`:

`./kafka-topics.sh --bootstrap-server localhost:9092 --list`

```
/opt/kafka/bin $ ./kafka-topics.sh --bootstrap-server localhost:9092 --list
david-carvajal
/opt/kafka/bin $
```

¿Sería posible crearlo en el entorno que tienes con un factor de replicación de 2?

¿Por qué?

[Introducción a Apache Kafka — AprenderBigData.com | by Oscar Fmdc | Medium](#)

Comando para crear un topic con particiones y factor de replicación:

`./kafka-topics.sh --bootstrap-server localhost:9092 --topic david-carvajal-replications --create --partitions 3 --replication-factor 2`

```
/opt/kafka/bin $ ./kafka-topics.sh --bootstrap-server localhost:9092 --topic david-carvajal-replications --create --partitions 3 --replication-factor 2
Error while executing topic command : Unable to replicate the partition 2 time(s): The target replication factor of 2 cannot be reached because only 1 broke
r(s) are registered.
[2025-05-05 18:38:09,086] ERROR org.apache.kafka.common.errors.InvalidReplicationFactorException: Unable to replicate the partition 2 time(s): The target re
plication factor of 2 cannot be reached because only 1 broker(s) are registered.
(org.apache.kafka.tools.TopicCommand)
/opt/kafka/bin $
```

Nos devuelve error porque solo tenemos un broker. Por tanto la respuesta es NO PORQUE solo tenemos un BROKER.

- Crea un productor que empiece a pasar información desde la línea de comandos al *topic* creado anteriormente y simular el envío de información.

Creación de un productor

`./kafka-console-producer.sh`

Este comando lleva a documentación, donde se pueden ver los parámetros obligatorios:

```

/opt/kafka/bin $ ./kafka-console-producer.sh
This tool helps to read data from standard input and publish it to Kafka.
Option                                     Description
-----
--batch-size <Integer: size>              Number of messages to send in a single
                                           batch if they are not being sent
                                           synchronously. please note that this
                                           option will be replaced if max-
                                           partition-memory-bytes is also set
                                           (default: 16384)
--bootstrap-server <String: server to    REQUIRED: The server(s) to connect to.
  connect to>                             The broker list string in the form
                                           HOST1:PORT1,HOST2:PORT2.
--compression-codec [String:             The compression codec: either 'none',
  compression-codec]                     'gzip', 'snappy', 'lz4', or 'zstd'.
                                           If specified without value, then it
                                           defaults to 'gzip'
--help                                    Print usage information.
--line-reader <String: reader_class>     The class name of the class to use for
                                           reading lines from standard in. By
                                           default each line is read as a
                                           separate message. (default: org.

```

Para crear el productor ejecutar lo siguiente:

```
./kafka-console-producer.sh --bootstrap-server localhost:9092 --topic david-carvajal
```

>escribir

>escribir

>escribir

CTRL-C para salir

```

/opt/kafka/bin $ ./kafka-console-producer.sh --bootstrap-server localhost:9092 --topic david-carvajal
>Hola
>clase
>que tal?
>^C/opt/kafka/bin $ |

```

- Creación de consumidores:
 - Crear un consumidor que sea capaz de leer toda la información que contenga el *topic* creado.

Creación de un consumidor

En este punto, vamos a ver cómo consumir la información que se ha dejado en el *topic* de Kafka.

Como siempre para ver la documentación:

```
./kafka-console-consumer.sh
```

```
>^C/opt/kafka/bin $ ./kafka-console-consumer.sh
This tool helps to read data from Kafka topics and outputs it to standard output.
Option                                     Description
-----
--bootstrap-server <String: server to    REQUIRED: The server(s) to connect to.
  connect to>
--consumer-property <String:              A mechanism to pass user-defined
  consumer_prop>                          properties in the form key=value to
                                           the consumer.
--consumer.config <String: config file>    Consumer config properties file. Note
                                           that [consumer-property] takes
                                           precedence over this config.
--enable-systest-events                   Log lifecycle events of the consumer
                                           in addition to logging consumed
                                           messages. (This is specific for
                                           system tests.)
--formatter <String: class>               The name of a class to use for
                                           formatting kafka messages for
                                           display. (default: org.apache.kafka.
                                           tools.consumer.
                                           DefaultMessageFormatter)
--formatter.config <String: config        Config properties file to initialize
  file>                                   the message formatter. Note that
                                           [property] takes precedence over
                                           this config.
--from-beginning                          If the consumer does not already have
                                           an established offset to consume
                                           from, start with the earliest
                                           message present in the log rather
                                           than the latest message.
--group <String: consumer group id>       The consumer group id of the consumer.
--help                                    Print usage information.
--include <String: Java regex (String)>   Regular expression specifying list of
```

Ejecutamos un consumidor:

Para leer todos los msgs, y no solo los que se manden una vez se ha arrancado, se aplica el parámetro ‘--from-beginning’.

```
./kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic
david-carvajal --from-beginning
```

```
/opt/kafka/bin $ ./kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic david-carvajal --from-beginning
Hola
clase
que tal?
```

Crear un grupo de consumidores con el suficiente número de ellos para que cada uno lea de una de las particiones del *topic*.

[Apache Kafka. Elementos y ejemplos con Python. - IABD](#)

[Kafka consumer group](#)

Primero crearemos un *topic* que contenga tres particiones:

```
./kafka-topics.sh --bootstrap-server localhost:9092 --topic david-carvajal-
partitions --create --partitions 3
```

```
/opt/kafka/bin $ ./kafka-topics.sh --bootstrap-server localhost:9092 --topic david-carvajal-partitions --create --partitions 3
Created topic david-carvajal-partitions.
```

Si comprobamos el estado del *topic* mediante:

```
./kafka-topics.sh --describe --topic david-carvajal-partitions --bootstrap-
server localhost:9092
```

Obtendremos la siguiente información:

```
/opt/kafka/bin $ ./kafka-topics.sh --describe --topic david-carvajal-partitions \  
> --bootstrap-server localhost:9092  
Topic: david-carvajal-partitions    TopicId: EINKH8LTR9i0pdBQRg.e9Q PartitionCount: 3      ReplicationFactor: 1    Configs: segment.bytes=1073741824  
    Topic: david-carvajal-partitions Partition: 0      Leader: 1      Replicas: 1      Isr: 1 Elr:      LastKnownElr:  
    Topic: david-carvajal-partitions Partition: 1      Leader: 1      Replicas: 1      Isr: 1 Elr:      LastKnownElr:  
    Topic: david-carvajal-partitions Partition: 2      Leader: 1      Replicas: 1      Isr: 1 Elr:      LastKnownElr:  
/opt/kafka/bin $
```

A continuación, en dos pestañas diferentes, vamos a crear dos consumidores que pertenezcan al mismo grupo de consumidores:

```
./kafka-console-consumer.sh --topic david-carvajal-partitions --group unir-app1 --  
bootstrap-server localhost:9092
```

```
opt/kafka/bin $ ./kafka-console-consumer.sh --topic david-carvajal-partitions \  
--group unir-app1 \  
--bootstrap-server localhost:9092
```

Ventana2 (DONDE DEBEMOS EJECUTAR EL EXEC para entrar en shell Kafka

```
./kafka-console-consumer.sh --topic david-carvajal-partitions --group unir-app1 --  
bootstrap-server localhost:9092
```

```
opt/kafka/bin $ ./kafka-console-consumer.sh --topic david-carvajal-partitions \  
--group unir-app1 \  
--bootstrap-server localhost:9092
```

Y finalmente, creamos un nuevo productor sobre el *topic*:

```
./kafka-console-producer.sh --topic david-carvajal-partitions --bootstrap-server  
localhost:9092 --property parse.key=true --property key.separator=,
```

Y escribimos lo siguiente (o podéis hacer otras mensajes

```
usuario1,Este es un mensaje para el usuario 1 productoA,Este es un mensaje sobre  
el producto A usuario2,Otro mensaje para el usuario 2 productoB,Información  
detallada del producto B usuario1,Un evento más para el usuario 1
```

```
/opt/kafka/bin $ ./kafka-console-producer.sh --topic david-carvajal-partitions \  
> --bootstrap-server localhost:9092 --property parse.key=true --property key.separator=,  
> usuario1,Este es un mensaje para el usuario 1  
productoA,Este es un mensaje sobre el producto A  
usuario2,Otro mensaje para el usuario 2  
productoB,Información detallada del producto B  
usuario1,Un evento más para el usuario 1>>>>  
>
```

```
Símbolo del sistema - docker x + v
/opt/kafka/bin $ ./kafka-console-consumer.sh --topic david-carvajal-partitions \
> --group unir-appl \
> --bootstrap-server localhost:9092

Este es un mensaje para el usuario 1
Este es un mensaje sobre el producto A
Un evento más para el usuario 1

/opt/kafka/bin $ ./kafka-console-consumer.sh --topic david-carvajal-partitions \
> --group unir-appl \
> --bootstrap-server localhost:9092

Otro mensaje para el usuario 2
Información detallada del producto B
```

Explicación de esta sentencia

- <host_kafka>:<puerto_kafka>: Reemplaza esto con la dirección y el puerto de tu broker de Kafka (por ejemplo, localhost:9092 o el nombre del servicio Kafka dentro de tu red Docker si estás usando Docker Compose, como kafka:9092).
- <nombre_del_topic>: Reemplaza esto con el nombre del topic al que quieres enviar mensajes.
- --property parse.key=true: Esta propiedad le dice al productor que espere un formato de entrada que incluya una clave.
- --property key.separator=,: Esta propiedad define el carácter que separará la clave del valor en tus mensajes. En este ejemplo, estamos usando una coma (,). Puedes usar cualquier otro carácter que desees (por ejemplo, :, -, etc.).

¿Cómo se escriben mensajes con clave y valor?

Después de ejecutar el comando, la consola esperará a que escribas los mensajes.

Debes escribir cada mensaje en la siguiente forma:

<clave>,<valor>

Y si creamos varios mensajes en el productor, veremos cómo van llegando de manera alterna a los diferentes consumidores:

Hay que tener en cuenta las siguientes consideraciones:

- **No especificar clave:** Si no especificas una clave de partición al producir mensajes, Kafka utilizará una estrategia de round-robin para distribuir los mensajes entre las particiones (y por lo tanto, entre los consumidores del grupo).
- **Usar claves diferentes:** Si necesitas mantener el orden de los mensajes relacionados, asegúrate de usar diferentes claves de partición para diferentes conjuntos de mensajes que no necesiten estar en el mismo orden. Una estrategia común es usar un ID de usuario o un ID de sesión como clave.